LABORATORY EXPERIMENTAL PROTOCOL - ANSWER SHEET

Applied Massive Parallelism: CUDA Python, Thread Geometries, and Hardware Latency Pitfalls

Student Name	Baiken Mukhamejanov
Student ID	230103062
Session / Date	Week 6 / 08.10.2026
Assigned GPU ID	Tesla T4 (Bus-Id 00000000:00:04.0)

SECTION 1: Hardware Telemetry (Task 1.1)

Field	Recorded value
NVIDIA Driver Version	580.82.07
CUDA Version Supported by Driver	13.0
Allocated GPU Name	Tesla T4
Total Available VRAM	15360 MiB
Compute Capability (Major.Minor)	7.5
Max Threads Per Block	1024

Additional: SMs = 40; Max Block Dimensions = 1024, 1024, 64; Warp Size = 32.

SECTION 2: Crossover Experiment (Tasks 2.1-2.2)

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner
100	0.061	1.178	0.090	CPU
1,000	0.541	0.702	0.082	CPU
10,000	5.203	0.690	0.078	GPU
100,000	51.714	0.860	0.091	GPU
1,000,000	540.001	5.497	0.143	GPU
5,000,000	skipped (nan)	16.158	0.296	GPU
10,000,000	skipped (nan)	26.342	0.466	GPU

Q1. Empirical crossover point N*:

N* = 10,000. At N = 1,000 the CPU still won (0.541 ms vs 0.702 ms GPU total), but at N = 10,000 the GPU total time (0.690 ms) became lower than the CPU time (5.203 ms) and stayed lower for all larger sizes.

Q2. Why is GPU Total so much slower than CPU at N = 100?

At N = 100 the GPU total time is 1.178 ms, while the kernel-only time is just 0.090 ms. So the actual computation is only about 7.6% of the elapsed time, and about 92% (1.088 ms) is overhead: allocating device memory, the Host-to-Device and Device-to-Host copies over the high-latency PCIe bus, and the kernel launch/synchronization cost. With only 100 floats the computation is trivial, so this fixed latency dominates, while the CPU loop finishes in 0.061 ms with no transfer at all.

SECTION 3: Thread Geometry & Boundary Corruption

Task 3.1 - Observed behavior / console output from defective kernel:

Launching defective kernel with 1024 threads on 1000-element array...
Execution finished without fatal crash. Processed elements: 1000. Last element: 999.0
No Python IndexError and no CUDA_ERROR_ILLEGAL_ADDRESS was raised. Threads 1000-1023 wrote out of bounds without any error, because GPU memory is raw VRAM with no automatic bounds checking; the 24 extra writes landed silently in memory beyond the array (undefined behavior that happened not to crash this time).

Task 3.2 - Thread coordinate inspector (N = 37, block size = 8):

Total Blocks Allocated	Total Hardware Threads Launched	Active Working Threads	Idle Guarded Threads
5	40	37	3

Boundary block 4: Global IDs 32-36 are ACTIVE; Global IDs 37, 38, 39 (Thread IDs 5, 6, 7) are IDLE (GUARDED).

SECTION 4: Race Condition (Parallel Reduction)

Trial #	Target Sum	Naive Kernel Output	Deviation / Lost Updates
Trial 1	50,000.0	2.0	49,998.0
Trial 2	50,000.0	2.0	49,998.0
Trial 3	50,000.0	2.0	49,998.0
Trial 4	50,000.0	2.0	49,998.0
Trial 5	50,000.0	2.0	49,998.0

Atomic Sum Output: 50000.0 - equals exactly 50,000.0? **Yes**

Performance penalty comparison: Naive Kernel Time: _____ ms vs. Atomic Kernel Time: _____ ms
(these two timings were not printed in the submitted notebook output)

Why atomic operations incur a penalty:
An atomic add makes the hardware lock the memory address and perform the read-modify-write as one indivisible step, so all 50,000 threads that target the same address, total[0], are serialized and executed one after another instead of in parallel. This removes the lost updates (naive result was only 2.0) but creates heavy contention on a single address and wastes the GPU's parallelism.

SECTION 5: 2D Grid Matrix Processing (1024 x 1024 radial vignette)

Blocks in X	Blocks in Y	Total Blocks	Pixel at Center (512, 512)	Pixel at Corner (0, 0)
64	64	4096	1.000	0.000

Output matrix shape: (1024, 1024); block size 16 x 16 = 256 threads; 1024/16 = 64 blocks per axis.

SECTION 6: Final Verification Token

TOKEN	AA83CD66B1CF4AE1
Inputs used	Student ID 230103062; GPU Tesla T4; CROSSOVER_N = 10000

Final checklist: notebook CUDA_Lab01_230103062.ipynb saved; all cells executed with visible output; exported as .ipynb; task sheet completed.